Phase 1: What A1–A4 Tell Us

Results of the four architecture runs, and the experiments they motivate

Manish Kumar (251330604)

CS-781, IIT Kanpur

27 September 2026

Outline

1. Setup of the four runs
2. Results
3. Next experiments
4. Plan

A controlled comparison: only the architecture changes

Shared by all four runs (`configs/base.yaml`)

- Legal byte-level BPE, 32k vocabulary.
- Mix: SC 30%, HC 35%, tribunals 25%, acts 10%.
- 3000 steps $\times$ 65,536 tokens = **196.6M tokens**.
- AdamW, lr $6\times10^{-4} \rightarrow 6\times10^{-5}$ cosine, 200 warm-up steps.
- Block 1024; validation loss on the 257 tokens after a 768-token prefix.
- Decoding: τ=0.8, top-k 50, top-p 0.95, rep. penalty 1.1.
- One NVIDIA H200 NVL.

Run	Design
A1	Decoder-only GPT: learned positions, LayerNorm, GELU, full causal MHA. 8L, d=768.
A2	Encoder–decoder: bidirectional encoder, cross-attending decoder, sinusoidal. 6+6L, d=640.
A3	Prefix-LM hybrid: bidirectional prompt + causal continuation, ALiBi. 8L, d=768.
A4	Efficient decoder: RoPE, RMSNorm, SwiGLU, GQA (4 KV heads), sliding window 256 with global layers 3/6/9. 10L, d=768.

Results at a glance

	Parameters		Validation		Dev set (200 prompts)			
Run	total	non-emb	loss	ppl	BERTScore	ROUGE-L	BLEU-4	GPU h
A1 GPT	82.1M	56.7M	**2.430**	**11.36**	**0.8364**	0.2225	**0.1104**	**0.39**
A2 Enc–dec	89.4M	68.9M	3.082	21.79	0.8154	0.1732	0.0318	0.87
A3 Prefix-LM	81.3M	56.7M	2.494	12.11	0.8349	0.2202	0.1100	0.41
A4 Efficient	87.5M	62.9M	2.445	11.53	0.8346	**0.2234**	**0.1104**	0.75

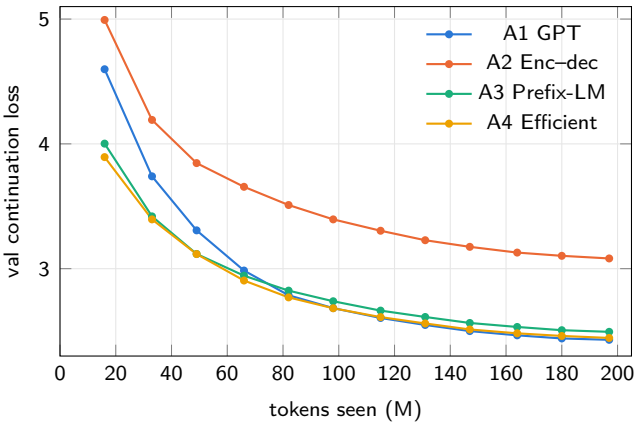
Ranking by validation loss

A1 $<$ A4 $<$ A3 $\ll$ A2. The three single-stack models are within 0.064 nats of each other. A2 is 0.65 nats behind.

Ranking by generation metrics

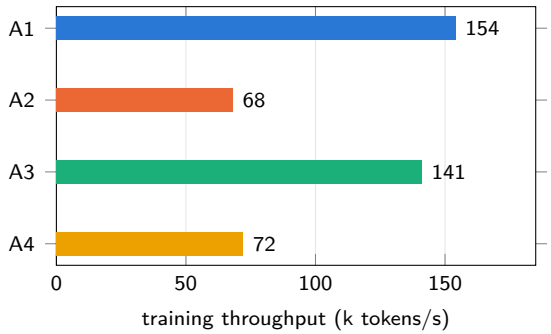
A2 is clearly worse: -0.021 BERTScore, and a third of the others' BLEU-4. A1, A3 and A4 differ by less than 0.002 BERTScore, which on 200 prompts is within sampling noise.

Validation curves: nobody has converged



- **A4 and A3 learn fastest early**: at 16M tokens A4 is 0.70 nats ahead of A1. RoPE/ALiBi and RMSNorm+SwiGLU speed up early learning.
- **A1 overtakes after about 100M tokens** and finishes best.
- **All curves still fall** in the last 16M tokens (A1 -0.011, A4 -0.017, A2 -0.021), even though the learning rate has already decayed to its floor.
- Budget: 196.6M tokens for 82M parameters is **2.4 tokens per parameter**, against the $\approx$20 that is compute-optimal (Chinchilla).

Efficiency: the GPU is mostly idle



Run	GPU util	peak alloc	energy
A1	35%	6.8 GB	0.093 kWh
A2	14%	5.4 GB	0.110 kWh
A3	36%	7.0 GB	0.097 kWh
A4	29%	8.4 GB	0.151 kWh

- The runs use **under 9 GB of a 141 GB card** and under 40% of its compute.
- A4 is "efficient" on paper but trains at **half the speed** of A1. The sliding window is applied as a mask over full attention, so it saves no FLOPs, and A4 has 10 layers rather than 8.
- Faster training means more tokens per GPU-hour.

What we learn from A1–A4

Findings

1. **Single-stack models win.** A2 splits its capacity: the decoder that writes the continuation has only 6 layers at d=640.
2. **A4's modern components do not pay off at 200M tokens**, even with 25% more depth. The early lead fades.
3. **The models are undertrained** (2.4 tokens/parameter; curves still falling).
4. **BERTScore is coarse.** A 0.65-nat gap (A2) costs 0.021 BERTScore, but a 0.06-nat gap (A1 vs A3) moves it by only 0.0015.

Generation problems seen on the test set

- Degenerate loops, for example "`.1.1.1.1`" in test ID 773, and repeated sentences.
- With `max_new_tokens`=256, 10 of 1000 continuations fell short of 128 words; 512 tokens left 1 short.

So what?

The largest gains are likely from **more tokens**, **faster training** and **better decoding**, not from further architecture changes. A1 and A4 are the two candidates to scale.

E1: train longer (the biggest lever)

Question: how much does loss keep falling with more tokens?

- Retrain A1 and A4 with `max_steps` = 6k, 12k and 24k (0.4B, 0.8B and 1.6B tokens). The cosine schedule must be *stretched*, not resumed: resuming at the 6×10^{-5} floor would waste the extra tokens.
- Set `train_tokens` to match so no data is repeated.
- Save intermediate checkpoints (every 2k steps) to use in E7.

```
python scripts/04_train.py -config
configs/a1_gpt.yaml \-set train.max_steps=12000
data.train_tokens=800_000_000 \run_name=A1_gpt_12k
```

Read-out

Plot val loss and dev BERTScore against tokens on a log axis. Fit $L(D) = E + B/D^{\beta}$ and extrapolate to decide the final budget.

Cost

A1 at 12k steps is about 1.6 GPU-h at today's speed, or well under 1 GPU-h after E2.

Hypothesis

A4's gated MLP and RoPE close the gap to A1, or overtake it, once data is no longer the bottleneck.

E2: make training faster before making it longer

Question: how much of the H200 can we actually use?

- `train.compile=true` (`torch.compile`).
- `micro_batch_size` 8→32 with `grad_accum_steps` 8→2. The tokens per step stay the same, so the results remain comparable.
- Use `scaled_dot_product_attention` (Flash) for every layer. Give A4's window layers a real banded kernel (FlexAttention) instead of a dense mask.
- Profile the data loader: memory-mapped token bins, `pin_memory`, more workers.

Read-out

Tokens/s, GPU utilisation and peak VRAM from `resources.csv`. Check that the loss curve *does not change*: this is an engineering experiment, not a modelling one.

Target

At least 2× throughput for A1 and 3× for A4, which makes E1 and E3 affordable.

E3: which A4 component helps? (one-at-a-time ablation)

A1 and A4 differ in five ways at once, so we cannot tell which change helped early and which hurt late. Starting from A1, add one A4 component per run (same data, same 3000 steps):

Run	Change from A1	What it isolates
E3a	learned positions $\rightarrow$ RoPE	positional scheme (fast early learning?)
E3b	LayerNorm $\rightarrow$ RMSNorm	normalisation (expected to be nearly neutral)
E3c	GELU 3072 $\rightarrow$ SwiGLU 2048	gated feed-forward at equal FLOPs
E3d	12 KV heads $\rightarrow$ 4 (GQA)	quality cost of the smaller KV cache
E3e	full attention $\rightarrow$ window 256 + global every 3	does local attention hurt long legal references?
E3f	8 layers $\rightarrow$ 10 layers at A1 width	depth versus the rest

Read-out: final val loss and loss at 50M tokens. Keep the components that help at *both* points in the scaled model. These are ablations of one design, not new architectures, so they count as analysis rather than towards the four-architecture requirement.

E4: use the parameter budget

Question: at a fixed GPU-hour budget, is it better to have more parameters or more tokens? The rules allow up to **300M**; we use 82M.

Size	d	layers	params
S (current)	768	8	82M
M	1024	12	$\approx$185M
L	1024	20	$\approx$286M

Train each at 2–3 token budgets so every size has an iso-FLOP point (IsoFLOP curves, Chinchilla approach 3).

Read-out

Val loss against FLOPs ($\approx 6ND$). Pick the size on the lower envelope for the final budget.

Watch

- `enforce_param_budget` must stay on.
- Larger models slow down inference in Phases 2–7 and need a lower peak lr (for example 3×10^{-4}).
- Model size is a scaling study, not a new architecture.

E5: decoding (cheap, and scored directly by BERTScore)

Question: which decoder gives the best dev BERTScore for a fixed checkpoint? Each point is one run of `05_generate.py` + `06_evaluate.py` on the 200 dev prompts (about 1 min on the H200).

- Greedy, and greedy + repetition penalty 1.1 / 1.2 / 1.3.
- $\tau \in \{0.3, 0.5, 0.7, 0.8, 1.0\} \times$ top-$p \in \{0.8, 0.9, 0.95\}$.
- Add **no-repeat** n**-gram** blocking (n=3, 4) to remove loops like "`.1.1.1`".
- Try min-p sampling and beam search (width 4) as extra baselines.
- Repeat the best 3 settings with 3 seeds to measure noise.

Why it matters

BERTScore and ROUGE reward text *close to the reference*. Lower-entropy decoding usually scores higher than creative sampling, while greedy decoding without penalties loops.

Also fix

Raise the default `max_new_tokens` to 512 and add a fallback when a sample still has fewer than 128 words, for example resampling that prompt with a different seed.

E6: data, tokenizer and context

Exp.	Controlled change	Read-out
E6a Data mix	Drop one source at a time; judgment-only mix; acts 10%→0%	val loss *per source*, dev BERTScore
E6b Match the test set	Estimate the source of each test prompt (acts vs judgments, SC vs HC) and match the training mix to it	dev BERTScore on a test-like dev set
E6c Tokenizer	Legal BPE 16k / 32k / 48k vs GPT-2 BPE	tokens per word, val loss *per word* (comparable across vocabularies)
E6d Context	Block 512 / 1024 / 2048; how much of the prompt is kept at decode time	val loss, dev BERTScore, speed
E6e Regularisation	Dropout 0.1 and more weight decay, but only once data is repeated (multi-epoch)	train–val gap

A trap to avoid

Different tokenizers produce different numbers of tokens for the same text, so compare loss **per word** (bits per word), not per token.

E7: does lower perplexity mean better text?

This is the key question in the brief. We have only 4 points so far: A2 is clearly separated, but the other three are tied on BERTScore.

- Score **every intermediate checkpoint** from E1 (for example 6 checkpoints $\times$ 2 models) and every E3 and E4 run on the dev set, all with the *same* decoder.
- Plot dev BERTScore / ROUGE-L against val loss and report Spearman ρ.
- Use bootstrap confidence intervals over the 200 dev prompts to show which differences are real.
- Grow the dev set to 500–1000 prompts if the intervals overlap.

Checkpoint selection

Compare `best.pt`, `last.pt` and an average of the last k checkpoints (or an EMA of the weights). Averaging is free and often gains a little.

Outcome

A decision rule for which run gets one of the limited Codabench submissions.

E8: distillation into a compact student

Question: can a smaller model keep most of the teacher's quality?

- Teacher: the best scaled model from E1/E4.
- Student: an A4-style model at about 50M parameters (the minimum allowed), using `configs/distill_efficient_small.yaml`.
- Compare three students: from scratch; logit KD (τ=2, α=0.5); KD plus a sweep over α.
- All three see the same tokens.

Read-out

Val loss, dev BERTScore, decoding tokens/s and VRAM. Report the share of the teacher's gain over the scratch student that KD recovers.

Why now

KD is **mandatory** in Phases 2–7. Getting the pipeline right in Phase 1 costs one short run.

Priorities and budget

#	Experiment	Why	GPU-h	Expected gain
1	E2 faster training	Makes every later run cheaper	0.5	none (enabler)
2	E5 decoding sweep	Directly moves the leaderboard metric; fixes loops	0.5	medium
3	E1 longer training (A1, A4)	Models are clearly undertrained	3–6	**high**
4	E7 perplexity vs BERTScore	The core analysis question; decides submissions	0.5	analysis
5	E3 A4 ablations	Explains A1 vs A4; picks final components	2	low–medium
6	E4 model size	Uses the unused 82M→300M budget	4–8	medium–high
7	E6 data / tokenizer / context	Domain fit to the test prompts	3	medium
8	E8 distillation	Rehearsal for Phases 2–7	1–2	compact model

GPU-hours are estimates at today's throughput; E2 should roughly halve them. A2 is dropped from scaling: it trails by 0.65 nats and trains at 44% of A1's speed.

Summary

Where we are

- Best model: **A1 GPT**, val loss 2.430 (ppl 11.36), dev BERTScore 0.8364.
- A4 is close (2.445); A3 is next (2.494); A2 is well behind (3.082).
- A1, A3 and A4 are tied on dev BERTScore.
- Every run is undertrained and uses a fraction of the GPU.

What next

1. Speed up training (E2).
2. Tune decoding (E5).
3. Train A1 and A4 for 4–8$\times$ more tokens (E1), then scale the winner (E4).
4. Tie val loss to BERTScore with confidence intervals (E7).
5. Distil the final model (E8).